Chapter 5: Unsupervised Learning and PCA

Part II — Introduction to Statistical Learning

Omid Safarzadeh

Abstract

Principal component analysis finds the few directions along which data vary the most, and throws the rest away. It is the workhorse of dimensionality reduction, and it has a striking double justification: the directions that *capture the most variance* are exactly the ones that *lose the least* when you reconstruct the data from them — and both are the top eigenvectors of the covariance matrix. This chapter sets up the projection, proves that equivalence, and connects it to the singular value decomposition that computes it in practice.

Contents

1 Directions of greatest variance 1

2 One answer to two questions 2

3 Singular values, how many components, and the fine print 3

1 Directions of greatest variance

Centre the data so each feature has mean zero, and collect it in an $n \times d$ matrix X with covariance $\Sigma = \frac{1}{n} X^\top X$. Project a point onto a unit direction w and you get the scalar $w^\top x$; the variance of those projections across the data is

$$\mathrm{Var}(w^\top x) = w^\top \Sigma\, w.$$

The **first principal component** is the unit direction that makes this as large as possible; the second is the best direction orthogonal to it, and so on. Projecting onto the top k components keeps the most variance any k directions could.

In practice you do not search directions by hand — the top component is the dominant eigenvector of Σ, which a few matrix-vector products find by **power iteration**:

```
def mat_vec(M, v):
    return [sum(M[i][j]*v[j] for j in range(len(v))) for i in range(len(M))]

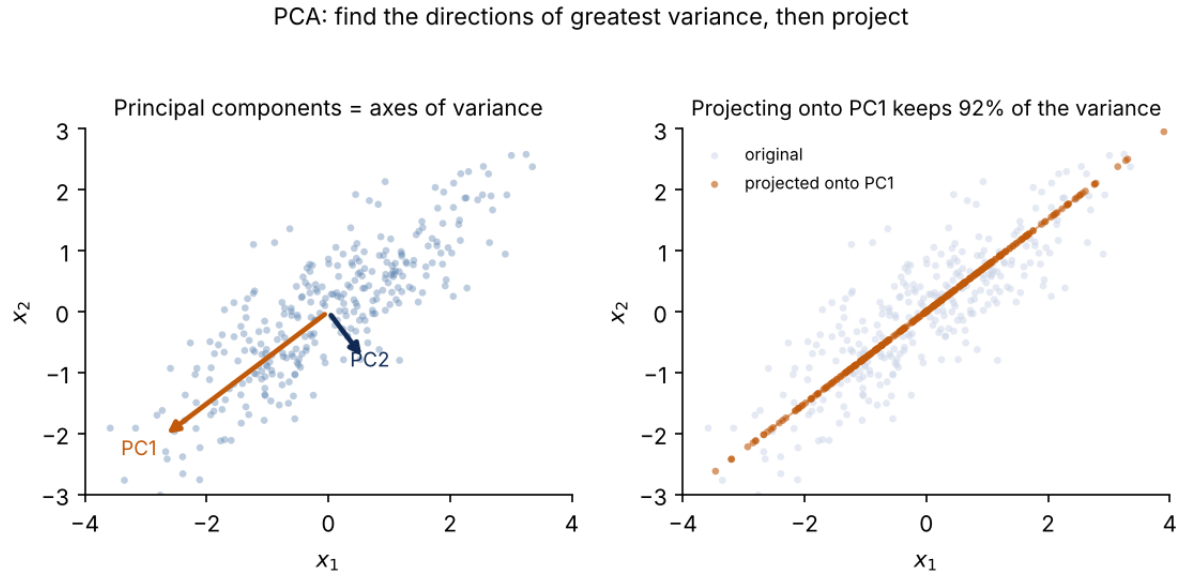
def normalize(v):
    s = sum(c*c for c in v) ** 0.5
    return [c/s for c in v]

def power_iteration(M, iters=300):          # top eigenvector of symmetric M
    v = normalize([1.0] + [0.0]*(len(M)-1))
    for _ in range(iters):
        v = normalize(mat_vec(M, v))
    lam = sum(vi*mi for vi, mi in zip(v, mat_vec(M, v)))   # Rayleigh quotient
    return lam, v

cov = [[2.0, 0.8], [0.8, 0.6]]
```

```
lam, pc1 = power_iteration(cov)
print(round(lam, 3))                    # 2.363  (largest eigenvalue)
print([round(c, 3) for c in pc1])       # [0.911, 0.413]  (PC1 direction)
```

The eigenvalue is the variance captured along that component. The figure shows the two principal directions of a correlated cloud and the projection onto the first — a single number per point that still holds most of the spread.



2 One answer to two questions

PCA can be posed two ways, and they give the same components.

Theorem 2.1 (Variance maximisation)**.** The unit vector w maximising $w^\top \Sigma w$ is the eigenvector of Σ with the largest eigenvalue, and the maximum value is that eigenvalue.

Proof. Maximise $w^\top \Sigma w$ subject to $w^\top w = 1$. The Lagrangian is $w^\top \Sigma w - \lambda(w^\top w - 1)$; setting its gradient to zero gives

$$\Sigma w = \lambda w,$$

so w is an eigenvector and λ its eigenvalue. At such a w the objective is $w^\top \Sigma w = \lambda$, so the maximum is attained at the *largest* eigenvalue. The spectral theorem guarantees Σ (symmetric, positive semidefinite) has an orthonormal eigenbasis, which gives the remaining components in order. □

Theorem 2.2 (Reconstruction-error minimisation)**.** Among all rank-k orthogonal projections $P = WW^\top$, the one minimising the reconstruction error $\sum_i \|x_i - Px_i\|^2$ is the projection onto the top k eigenvectors of Σ — the same components.

Sketch. By the Pythagorean theorem $\|x\|^2 = \|Px\|^2 + \|x - Px\|^2$, so minimising the reconstruction error $\sum_i \|x_i - Px_i\|^2$ is the same as maximising the retained energy $\sum_i \|Px_i\|^2 = \operatorname{tr}(W^\top \Sigma W)$. That trace is maximised, over orthonormal W, by the top k eigenvectors (a direct consequence of the first theorem applied one direction at a time). □

So "keep the most variance" and "lose the least information" are not two heuristics that happen to agree — they are the same optimisation, and its solution is the eigenvectors of the covariance

matrix.

3 Singular values, how many components, and the fine print

In practice PCA is computed from the **singular value decomposition** $X = UDV^\top$, not by forming Σ. The right singular vectors (columns of V) are the principal components, and the singular values relate to the eigenvalues of Σ by $\sigma_j^2 = n\lambda_j$. Working from X directly is more numerically stable than squaring it into $X^\top X$.

How many components to keep is read from the eigenvalues: the fraction of variance explained by component j is $\lambda_j / \sum_\ell \lambda_\ell$, and you keep enough that the cumulative fraction crosses a threshold — the "elbow" of the **scree plot**. The error you accept is exactly the variance you drop, $\sum_{j>k} \lambda_j$.

Two cautions keep PCA honest. It is *linear*: it finds flat subspaces, so data lying on a curved manifold needs **kernel PCA** or a nonlinear method. And it equates variance with importance, so features must be put on a common scale first (standardise, or PCA just tracks whichever feature has the biggest units). Within those limits, PCA is the first thing to reach for when there are more dimensions than you can look at.

Summary

PCA projects onto the directions of greatest variance, which are the top eigenvectors of the covariance Σ — found by power iteration or, in practice, the SVD. The components maximise retained variance $w^\top \Sigma w$ and, equivalently, minimise reconstruction error; both reduce to $\Sigma w = \lambda w$, so the two views give one answer. The variance explained by component j is $\lambda_j / \sum_\ell \lambda_\ell$, which is how you choose how many to keep. PCA is linear and scale-sensitive, so standardise first.